INSTITUTO FEDERAL GOIANO — CAMPUS IPORÁ
Bacharelado em Ciência da Computação • 3º Período — 2026/2

Teoria dos Grafos — Lista de Exercícios 3: Caminhos Mínimos

Tópicos: Dijkstra, Bellman-Ford e Floyd-Warshall | **Docente:** Prof. Newarney T. Costa
Integrantes da Dupla: Messias Junio da Silva Mendes & Bruno Alves Mendonça De Abreu Filho
Adaptação Acadêmica: Resolução integralmente digitada em conformidade com as diretrizes institucionais de acessibilidade funcional.

Questão 1 — Algoritmo de Dijkstra (Um-para-Todos)

Grafo não orientado com pesos (vértices a, b, c, d, e, f): a–b(4), a–c(2), b–c(1), b–d(5), c–d(8), c–e(10), d–e(2), d–f(6), e–f(3).

1.1 Funcionamento, Características e Limitações (Pesos Negativos)

- **Como o Dijkstra funciona:** O algoritmo de Dijkstra determina a distância mínima de um vértice de origem para todos os demais vértices mantendo uma estimativa `dist[v]` (inicialmente 0 para a origem e ∞ para os demais) e dividindo os vértices em dois conjuntos: *abertos* (ainda não definitivos) e *fechados* (cuja menor distância já foi fixada). Em cada passo, escolhe de forma **gulosa** o vértice aberto u com a menor distância provisória, fecha u e realiza o **relaxamento** de todas as arestas `(u, v)` para vizinhos abertos: se `dist[u] + peso(u, v) < dist[v]`, atualiza `dist[v] = dist[u] + peso(u, v)` e define `antecessor[v] = u`.
- **Principais características:** Estratégia gulosa que fecha exatamente um vértice por iteração e nunca o reabre; possui complexidade **O(V²)** usando matriz de adjacência e busca linear, ou **O((V + E) log V)** usando lista de adjacência e Fila de Prioridade (Min-Heap).
- **Limitação quanto a arestas de peso negativo:** O Dijkstra **não funciona** quando existem arestas de peso negativo. Isso ocorre porque, ao fechar o vértice aberto u de menor distância atual, o algoritmo assume que qualquer outro caminho passando por vértices ainda abertos só poderá somar pesos positivos ou nulos, nunca ficando menor que `dist[u]`. Se houver uma aresta negativa mais adiante no grafo, um caminho inicialmente mais caro pode sofrer um desconto negativo e tornar-se menor do que a distância de um vértice que o Dijkstra já fechou, sem que o algoritmo consiga corrigi-lo.

1.2 Aplicação Manual do Dijkstra (Tabela Iteração a Iteração)

Formato de cada célula: `Distância | Fechado (*) | Antecessor`.

Vértice	Inicial	Iter. 1 (Fecha a)	Iter. 2 (Fecha c)	Iter. 3 (Fecha b)	Iter. 4 (Fecha d)	Iter. 5 (Fecha e)	Iter. 6 (Fecha f)
a (0)	0 | [ ] | -	**0 | [*] | -**	0 | [*] | -	0 | [*] | -	0 | [*] | -	0 | [*] | -	0 | [*] | -
b (1)	∞ | [ ] | -	4 | [ ] | a	3 | [ ] | c	**3 | [*] | c**	3 | [*] | c	3 | [*] | c	3 | [*] | c
c (2)	∞ | [ ] | -	2 | [ ] | a	**2 | [*] | a**	2 | [*] | a	2 | [*] | a	2 | [*] | a	2 | [*] | a
d (3)	∞ | [ ] | -	∞ | [ ] | -	10 | [ ] | c	8 | [ ] | b	**8 | [*] | b**	8 | [*] | b	8 | [*] | b
e (4)	∞ | [ ] | -	∞ | [ ] | -	12 | [ ] | c	12 | [ ] | c	10 | [ ] | d	**10 | [*] | d**	10 | [*] | d
f (5)	∞ | [ ] | -	∞ | [ ] | -	∞ | [ ] | -	∞ | [ ] | -	14 | [ ] | d	13 | [ ] | e	**13 | [*] | e**

Recuperação do Caminho Mínimo de *a* até *f*: Seguindo os antecessores a partir de *f*: `ant[f] = e → ant[e] = d → ant[d] = b → ant[b] = c → ant[c] = a`.
Caminho: a → c → b → d → e → f | **Custo Mínimo Total:** 2 + 1 + 5 + 2 + 3 = **13**.

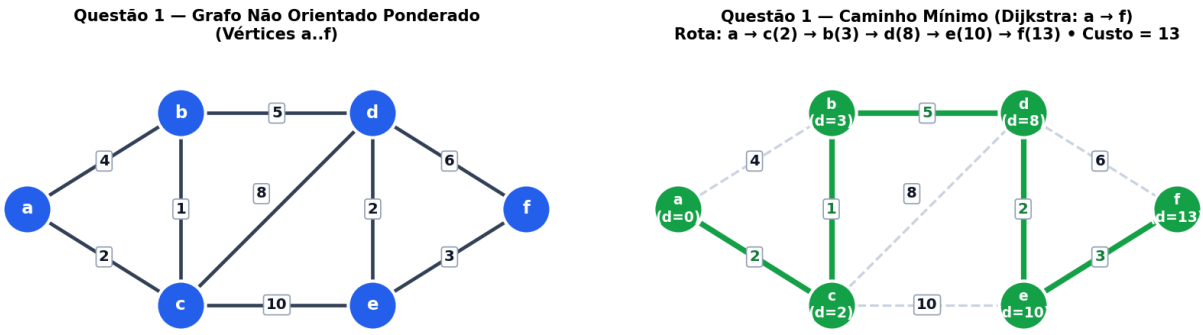


Figura 1 — Grafo não orientado da Questão 1 e o caminho mínimo de a até f (custo = 13) obtido pelo Algoritmo de Dijkstra.

1.3 Implementação Completa (Matriz Simétrica e Relaxamento em Python e C)

```
# Python (Google Colab) — Matriz Simétrica e Relaxamento do Dijkstra
grafo = [
    [0, 4, 2, 0,  0, 0], # a (0)
    [4, 0, 1, 5,  0, 0], # b (1)
    [2, 1, 0, 8, 10, 0], # c (2)
    [0, 5, 8, 0,  2, 6], # d (3)
    [0, 0, 10, 2, 0, 3], # e (4)
    [0, 0, 0, 6,  3, 0]  # f (5)
]
# Condição de relaxamento dentro de for v in range(V):
if not visitado[v] and grafo[u][v] > 0 and dist[u] != sys.maxsize and dist[u] + grafo[u][v] < dist[v]:
    dist[v] = dist[u] + grafo[u][v]

/* Em C (dentro do laço for (int v = 0; v < V; v++)): */
if (!visitado[v] && grafo[u][v] && dist[u] != INT_MAX && dist[u] + grafo[u][v] < dist[v]) {
    dist[v] = dist[u] + grafo[u][v];
}
```

Questão 2 — Algoritmo de Bellman-Ford (Pesos Negativos)

Grafo orientado com pesos (vértices A..F): A→B(4), A→C(2), B→C(−1), B→D(2), C→D(3), C→E(2), D→F(2), E→D(−3), E→F(3).

2.1 Pesos Negativos, Detecção de Ciclo Negativo e Diferença para o Dijkstra

- **Como lida com pesos negativos:** Em vez de fechar vértices definitivamente, o Bellman-Ford relaxa **todas as E arestas do grafo em V − 1 iterações completas**. Como qualquer caminho simples sem ciclos em um grafo com V vértices possui no máximo V − 1 arestas, a k-ésima iteração garante o custo ótimo de todos os caminhos com até k arestas, permitindo que reduções geradas por arcos negativos (como E→D com peso −3) sejam propagadas para os vértices seguintes nas rodadas subsequentes.
- **Condição que identifica um ciclo de peso negativo:** Após as V − 1 iterações, realiza-se uma V-ésima verificação sobre todas as arestas `(u, v)`. Se ainda existir alguma aresta tal que `dist[u] != INF && dist[u] + peso(u, v) < dist[v]`, significa que a distância continua diminuindo após V − 1 arestas, comprovando a existência de um **ciclo de peso negativo** alcançável a partir da origem.
- **Diferença principal para o Dijkstra:** O Dijkstra é um algoritmo *guloso* (avalia cada aresta apenas quando seu vértice de origem é fechado, em tempo O(V²) ou O((V+E) log V), mas falha com pesos negativos), enquanto o Bellman-Ford usa *programação dinâmica / relaxamento global* (reavalia todas as E arestas V − 1 vezes, em tempo **O(V · E)**, funcionando com pesos negativos e detectando ciclos negativos).

2.2 Aplicação Manual do Bellman-Ford a partir de A

Relaxando os 9 arcos na ordem `(A→B:4)`, `(A→C:2)`, `(B→C:-1)`, `(B→D:2)`, `(C→D:3)`, `(C→E:2)`, `(D→F:2)`, `(E→D:-3)`, `(E→F:3)`:

Vértice	Inicial	Após Iteração 1	Após Iteração 2	Iterações 3, 4 e 5	Check de Ciclo Negativo
A (0)	0 | -	0 | -	0 | -	0 | -	Nenhuma redução
B (1)	∞ | -	4 | A	4 | A	4 | A	Nenhuma redução
C (2)	∞ | -	2 | A *(B→C dá 4-1=3 > 2)*	2 | A	2 | A	Nenhuma redução
D (3)	∞ | -	**1 | E** *(via E→D: 4-3=1)*	1 | E	1 | E	Nenhuma redução
E (4)	∞ | -	4 | C *(via C→E: 2+2=4)*	4 | C	4 | C	Nenhuma redução
F (5)	∞ | -	7 | D *(quando D valia 5)*	**3 | D** *(via D→F: 1+2=3)*	3 | D	Nenhuma redução

Conclusão sobre Ciclo Negativo: Não existe ciclo de peso negativo (a partir da 2ª iteração todas as distâncias estabilizam em `[0, 4, 2, 1, 4, 3]`). Caminho mínimo de A até F: **A → C → E → D → F** (custo = 2 + 2 − 3 + 2 = **3**).

2.3 Implementação Completa do Bellman-Ford (Python e C)

```
# Python (Google Colab)
def bellman_ford(arestas, origem):
    dist = [float('inf')] * V
    dist[origem] = 0
    for _ in range(V - 1):
        for u, v, peso in arestas:
            if dist[u] != float('inf') and dist[u] + peso < dist[v]:
                dist[v] = dist[u] + peso
    for u, v, peso in arestas:
        if dist[u] != float('inf') and dist[u] + peso < dist[v]:
            print("Ciclo de peso negativo detectado!")
            return
    for i in range(V):
        print(f"Distancia de {origem} ate {i} = {dist[i]}")

/* Em C (V-1 iterações e verificação de ciclo negativo): */
for (int i = 1; i <= V - 1; i++)
    for (int j = 0; j < E; j++)
        if (dist[arestas[j].origem] != INT_MAX && dist[arestas[j].origem] + arestas[j].peso < dist[arestas[j].destino])
            dist[arestas[j].destino] = dist[arestas[j].origem] + arestas[j].peso;
```

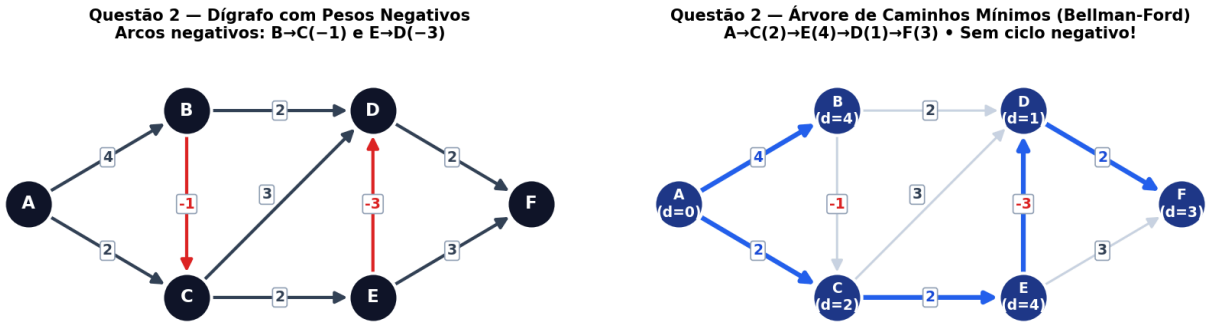


Figura 2 — Dígrafo com pesos negativos da Questão 2 e a árvore de caminhos mínimos obtida pelo Bellman-Ford.

Questão 3 — Algoritmo de Floyd-Warshall (Todos os Pares)

Grafo orientado com pesos (vértices A..F): A→B(4), A→C(2), B→C(3), B→D(2), C→D(4), C→E(5), D→E(1), E→F(2), D→F(6).

3.1 Estrutura (Distâncias e Roteamento), Atualização por k e Complexidade

- **Estrutura:** Utiliza duas matrizes `V × V`: a **Matriz de Distâncias D** (onde `dist[i][j]` guarda o custo do menor caminho de `i` para `j`) e a **Matriz de Roteamento R** (onde `prox[i][j]` guarda o próximo vértice após `i` no caminho

mínimo até `j`, ou `pred[i][j]` guarda o antecessor de `j`).

- **Atualização a cada vértice intermediário k:** Na iteração k (de `0` a `V − 1`), o algoritmo verifica para todo par `(i, j)` se passar pelo vértice intermediário k reduz a distância conhecida:
  `if (dist[i][k] + dist[k][j] < dist[i][j]) { dist[i][j] = dist[i][k] + dist[k][j]; prox[i][j] = prox[i][k]; }`
- **Complexidade de Tempo e Espaço:** Como são 3 laços `for` aninhados (`k`, `i`, `j`) variando de `0` a `V − 1` com operações O(1) no interior, a complexidade de tempo é **Θ(V³)** e a de espaço é **Θ(V²)**.

3.2 Evolução das Matrizes D⁽ᵏ⁾ Passo a Passo e Caminho de A até F

- **Matriz Inicial D⁽⁰⁾ e k = 0 (A):** Nenhuma distância muda em `k = A`, pois A não possui arcos de entrada.
- **Após k = 1 (B):** O caminho `A → B → D` atualiza `dist[A][D]` de ∞ para `4 + 2 = 6`.
- **Após k = 2 (C):** O caminho `A → C → E` atualiza `dist[A][E]` de ∞ para `2 + 5 = 7`; `B → C → E` atualiza `dist[B][E]` para `3 + 5 = 8` (e `A → C → D = 2 + 4 = 6` empata com `dist[A][D] = 6`).
- **Após k = 3 (D):** Atualizam-se: `dist[A][F] = dist[A][D] + dist[D][F] = 6 + 6 = 12`; `dist[B][E] = dist[B][D] + dist[D][E] = 2 + 1 = 3` (melhora de 8 para 3); `dist[B][F] = 2 + 6 = 8`; e `dist[C][F] = 4 + 6 = 10`.
- **Após k = 4 (E) e k = 5 (F) (Matriz Final D⁽⁶⁾):** Passando por E como intermediário (já que `E → F = 2`), reduzimos todas as chegadas em F:
  - `dist[A][F]` cai de 12 para `dist[A][E] + dist[E][F] = 7 + 2 = 9`;
  - `dist[B][F]` cai de 8 para `dist[B][E] + dist[E][F] = 3 + 2 = 5`;
  - `dist[C][F]` cai de 10 para `dist[C][E] + dist[E][F] = 5 + 2 = 7`;
  - `dist[D][F]` cai de 6 para `dist[D][E] + dist[E][F] = 1 + 2 = 3`.

Matriz Final de Distâncias D⁽⁶⁾ e Matriz de Roteamento (Sucessores R):

```
Matriz Final de Distâncias D(6):
      A    B    C    D    E    F
A [   0,   4,   2,   6,   7,   9 ]
B [ INF,   0,   3,   2,   3,   5 ]
C [ INF, INF,   0,   4,   5,   7 ]
D [ INF, INF, INF,   0,   1,   3 ]
E [ INF, INF, INF, INF,   0,   2 ]
F [ INF, INF, INF, INF, INF,   0 ]
```

```
Matriz de Roteamento (Próximo Vértice no Caminho):
      A    B    C    D    E    F
A [   -,   B,   C,   B,   C,   C ]
B [   -,   -,   C,   D,   D,   D ]
C [   -,   -,   -,   D,   E,   E ]
D [   -,   -,   -,   -,   E,   E ]
E [   -,   -,   -,   -,   -,   F ]
F [   -,   -,   -,   -,   -,   - ]
```

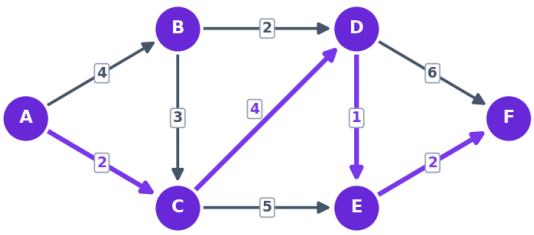
Menor distância de A até F e caminho correspondente:

• **Menor distância:** `dist[A][F] = 9`.

• **Caminho recuperado pela Matriz de Roteamento:** `prox[A][F] = C → prox[C][F] = E → prox[E][F] = F` ⇒ **A → C → E → F** (2 + 5 + 2 = 9).

*(Nota: Os caminhos **A → C → D → E → F** (2+4+1+2 = 9) e **A → B → D → E → F** (4+2+1+2 = 9) possuem exatamente o mesmo custo ótimo 9).*

Questão 3 — Dígrafo (Floyd-Warshall)
Distância Mínima A → F = 9 (ex: A→C→D→E→F)



Matriz Final de Distâncias Mínimas D⁽⁶⁾
(Célula destacada: dist[A][F] = 9)

	A (0)	B (1)	C (2)	D (3)	E (4)	F (5)
A (0)	0	4	2	6	7	9
B (1)	∞	0	3	2	3	5
C (2)	∞	∞	0	4	5	7
D (3)	∞	∞	∞	0	1	3
E (4)	∞	∞	∞	∞	0	2
F (5)	∞	∞	∞	∞	∞	0

Figura 3 — Dígrafo da Questão 3 e a Matriz Final de Distâncias Mínimas entre todos os pares (Floyd-Warshall).

Questão 4 — Comparação: Dijkstra × Bellman-Ford × Floyd-Warshall

Critério	Dijkstra	Bellman-Ford	Floyd-Warshall
1. Subproblema Resolvido	**Um-para-todos** (Fonte Única — SSSP)	**Um-para-todos** (Fonte Única — SSSP)	**Todos os pares** (APSP)
2. Ordem de Complexidade O	• Matriz/Vetor: **$O(V^2)$** • Min-Heap: **$O((V + E) \log V)$**	**$O(V \cdot E)$** *(No caso denso $E \approx V^2$, $O(V^3)$)*	Tempo: **$\Theta(V^3)$** Espaço: **$\Theta(V^2)$**
3. Pesos Negativos e Ciclos	**Não aceita pesos negativos** (pode falhar ao fechar vértices prematuramente).	**Aceita pesos negativos** e **detecta ciclos negativos** alcançáveis da origem.	**Aceita pesos negativos** e identifica ciclos negativos se `dist[i][i] < 0` na diagonal.
4. Vantagens e Limitações (Relação com Q1–Q3)	Na **Questão 1** (pesos ≥ 0), foi o mais direto: fechou cada um dos 6 vértices uma única vez sem precisar reavaliar arestas já fechadas.	Na **Questão 2**, graças às rodadas de relaxamento, propagou corretamente o desconto do arco `E→D(−3)` até F na 2ª iteração, onde o Dijkstra falharia se fechasse D antes de E.	Na **Questão 3**, com apenas 3 laços aninhados, entregou simultaneamente a menor distância entre todos os 36 pares de vértices e permitiu recuperar qualquer rota via matriz de roteamento.